

RateForge

Rate Limiting Experimentation & Analysis Tool

PROJECT REPORT

Programming in Java

VIT Bhopal University

Name - Kartikey Srivastava

Roll No. - 25BAI11235

Course - Programming in Java

Professor - Dr. Muthusundari S.

Contents :

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-functional Requirements
5. System Architecture
6. Design Diagrams
7. Design Decisions & Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Conclusion

1. Introduction :

Rate limiting is a common technique used in software systems to control the number of requests that can be accepted in a given period of time. It is useful when a system needs to protect resources from excessive traffic or prevent one source from generating too many requests.

RateForge is a Java CLI application built to study this concept through simulation. Instead of sending real network requests, the application creates simulated request timestamps and passes the same workload through different rate-limiting algorithms.

The project currently implements *Fixed Window*, *Sliding Window*, *Token Bucket*, and *Leaky Bucket*. Users can select an individual algorithm, compare all algorithms, run live simulations, use predefined traffic scenarios, learn about the algorithms, and export the latest results to CSV.

Project Objectives

- Implement four common rate-limiting approaches in Java.
 - Generate controllable traffic instead of relying on a fixed request list.
 - Allow the same workload to be tested against different algorithms.
 - Show accepted and rejected requests and basic performance measurements.
 - Provide an interactive CLI that can be used without a graphical interface.
 - Create a practical project for understanding algorithm behaviour.
-

2. Problem Statement :

Applications and APIs may receive many requests in a short period of time. If the incoming traffic is not controlled, excessive requests can consume system resources and affect service availability or fairness between users.

Different rate-limiting algorithms solve this problem using different mechanisms. A simple counter, a rolling timestamp window, a token bucket, and a leaky bucket can produce different results when the traffic pattern changes.

The problem addressed by RateForge is to create a simple environment where these algorithms can be implemented and tested under controlled traffic conditions. The application should make the behaviour visible instead of only providing theoretical descriptions.

Scope

The scope is limited to simulation and comparison. It does not attempt to act as a production API gateway or distributed rate-limiting service.

- Algorithm implementation and simulation.
 - Configurable traffic generation.
 - Single-algorithm and multi-algorithm experiments.
 - Live request-level observation.
 - Basic result analysis and CSV export.
 - Command-line operation.
-

3. Functional Requirements :

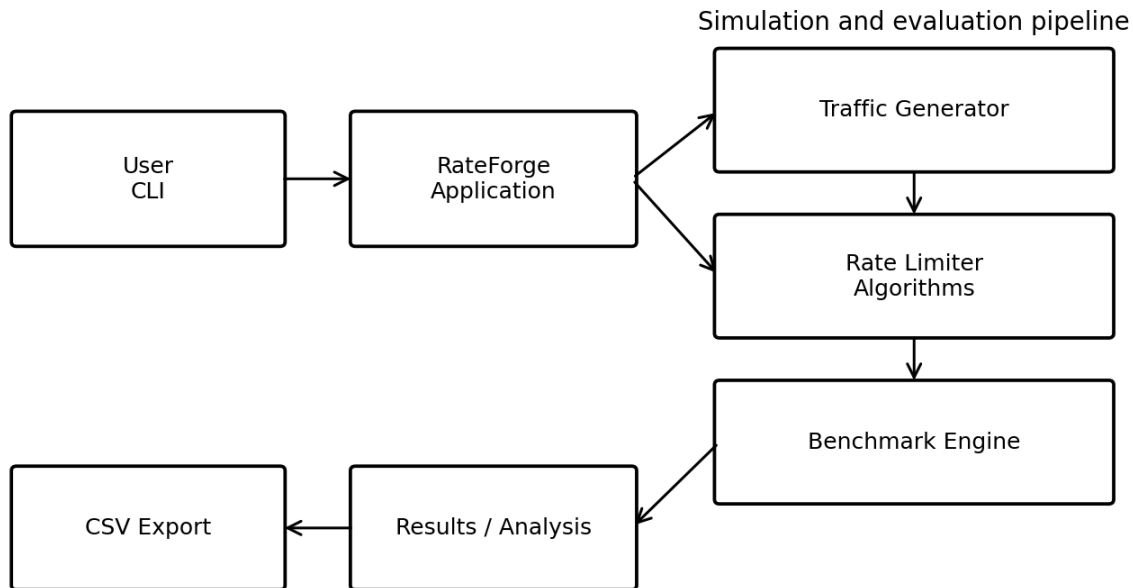
S.No.	Requirement	Description
1	Interactive menu	The system shall provide a menu for selecting major application functions.
2	Algorithm selection	The user shall be able to select Fixed Window, Sliding Window, Token Bucket, or Leaky Bucket.
3	Experiment configuration	The system shall accept a rate limit, duration, base rate, peak rate, and traffic pattern.
4	Traffic generation	The system shall generate simulated requests from the configured traffic conditions.
5	Algorithm comparison	The system shall run the same generated request stream through all four algorithms.
6	Live simulation	The system shall display individual request decisions and algorithm state.
7	Presets	The system shall provide predefined traffic experiments.
8	Learning mode	The system shall provide basic information about the algorithms.
9	Results	The system shall display accepted, rejected, and percentage results.
10	CSV export	The system shall export the latest benchmark results to a CSV file.
11	CLI execution	The system shall support execution from a terminal with command-line arguments.
12	Validation	The system shall detect invalid input and report errors.

4. Non-functional Requirements :

Requirement	Description
Usability	The CLI should have clear menus, labels, and readable result tables.
Performance	The simulator should process generated events efficiently using simple in memory structures.
Portability	The project should run on systems with a compatible Java JDK without external dependencies.
Maintainability	Algorithm logic should be separated through a common Rate limiter interface and supporting classes.
Reliability	Invalid configuration and common input errors should be handled without unexpected termination in normal interactive use.
Reproducibility	Comparison experiments should use the same generated request stream for the algorithms being compared.
Simplicity	This implementation should remain small enough to compile and understand as a course project.

5. System Architecture :

RateForge follows a simple layered flow. The command-line interface collects the experiment settings. The traffic generator creates simulated requests. One or more rate limiter implementations process those requests. The benchmark engine collects the decisions and produces results for display or CSV export.



High-level RateForge architecture.

The RateLimiter interface provides a common contract for all four algorithms. This allows the benchmark engine to work with any algorithm without duplicating the experiment logic.

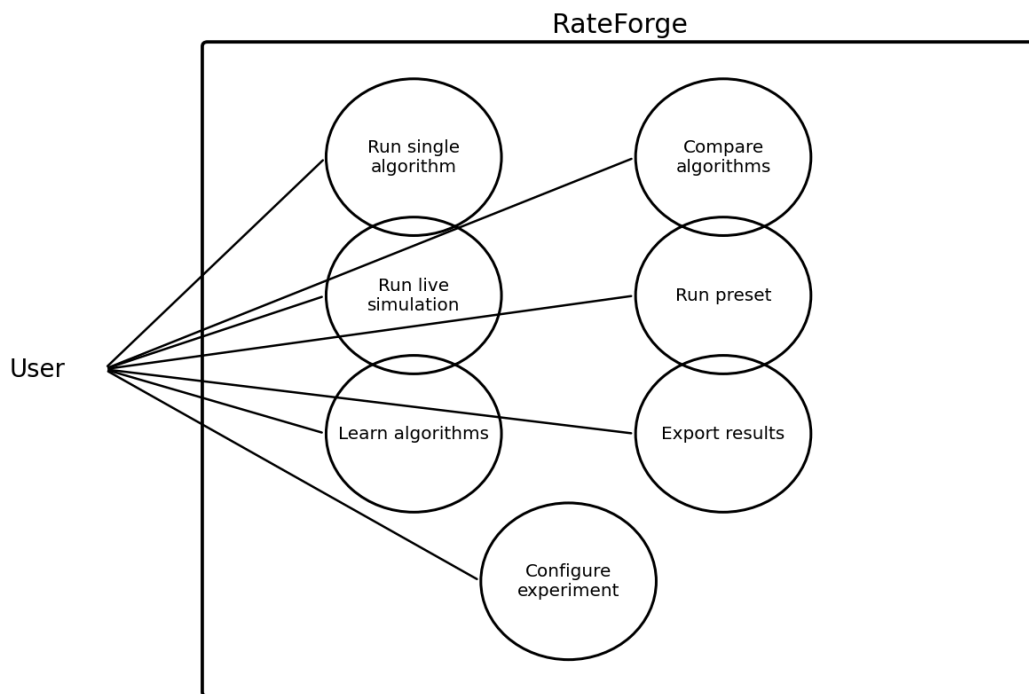
Main Components

Component	Purpose
CLI / Application logic	Handles menus, command-line options, user input and program flow.
RateLimiter	Common interface used by every rate-limiting algorithm.
Rate limiter implementations	Contain the individual algorithm logic.
TrafficGenerator	Creates simulated request timestamps for different traffic patterns.
SimulatedRequest	Stores a request ID and simulated timestamp.
BenchmarkEngine	Runs requests through a limiter and calculates results.
BenchmarkResult	Stores accepted, rejected, percentages and processing time.
Configuration	Stores the current experiment settings.

6. Design Diagrams :

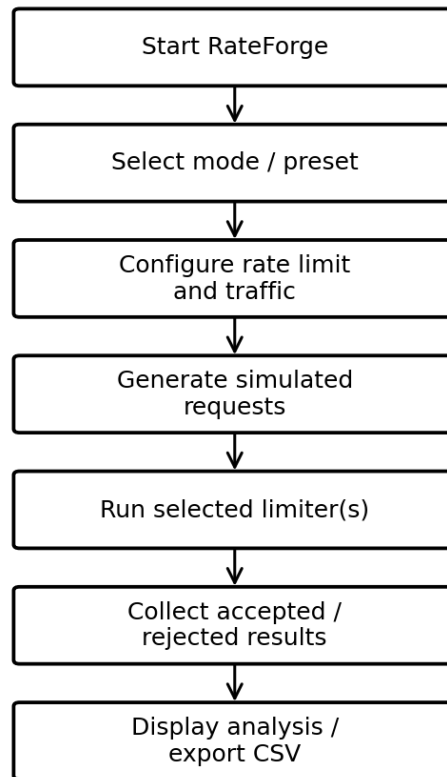
6.1 Use Case Diagram

The main actor is the user operating RateForge from the command line. The user can configure and run experiments, compare algorithms, inspect individual requests, learn about algorithms, and export results.



Use case diagram.

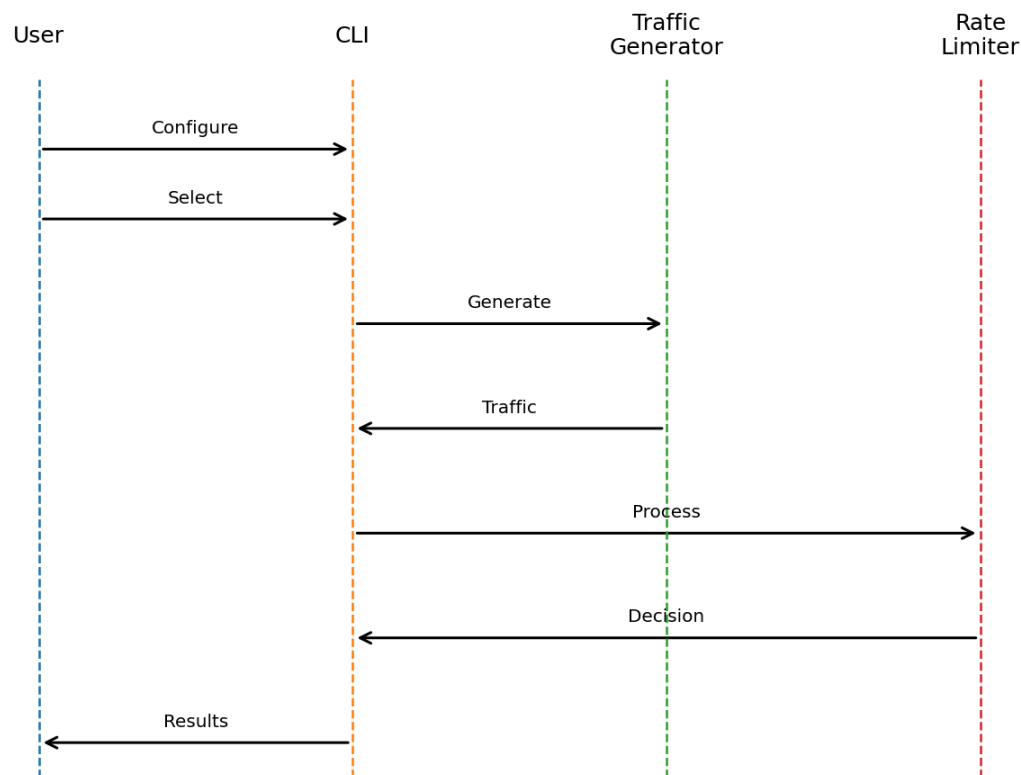
6.2 Workflow Diagram



Main experiment workflow.

6.3 Sequence Diagram

The sequence below shows a simplified custom comparison experiment. The user configures the experiment, the CLI asks the traffic generator for simulated requests, and the benchmark engine processes those requests using the selected limiter.

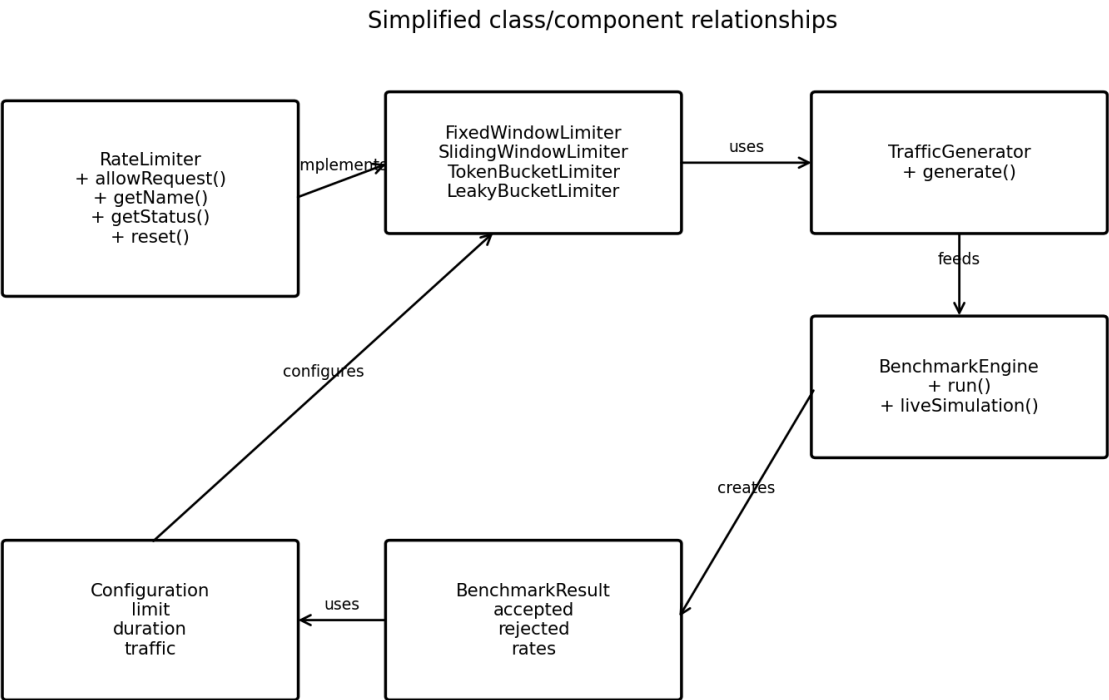


Simplified sequence of a custom comparison experiment

Simplified experiment sequence.

6.4 Class / Component Diagram

The implementation uses a RateLimiter interface and four concrete limiter classes. Supporting classes handle traffic generation, requests, benchmarking, results, and configuration.



Simplified class/component relationships.

6.5 ER Diagram

An ER diagram is not applicable to this project because the application does not use a database or persistent relational storage. Experiment data exists in memory during execution, and the optional CSV file is an output file rather than a database.

Storage	Used	Reason
Relational database	No	Not required for the current simulation scope.
Redis / external store	No	No distributed or persistent state is needed for now
CSV output	Optional	Used only to export the latest experiment results.

7. Design Decisions & Rationale :

Command-line interface: The project is intended to be fully executable from the terminal and a CLI makes the experiments easy to run without adding GUI complexity.

Common RateLimiter interface: A common interface allows the benchmark engine to treat all algorithms uniformly and demonstrates polymorphism.

Simulated timestamps: Using simulated time makes experiments faster and repeatable compared with waiting for real seconds.

Configurable traffic rates: Traffic is generated from rates and duration so the user can create normal, overload, burst, periodic, and random scenarios.

Same workload for comparison: All algorithms receive the same generated request stream, reducing differences caused by separate traffic generation.

In-memory data structures: The current scope does not need a database. Java collections are enough for the simulation.

Single-file : Keeping the prototype in one file makes it easy to compile and submit. It can be split into packages in a later version.

No external frameworks: The main goal is to demonstrate Java, algorithms, data structures, and OOP rather than framework configuration.

8. Implementation Details :

8.1 Fixed Window

The FixedWindowLimiter stores the beginning of the current window and the number of accepted requests in that window. When the timestamp moves into a new window, the counter is reset.

8.2 Sliding Window

The SlidingWindowLimiter stores accepted request timestamps in an ArrayDeque. Before checking a new request, timestamps older than the configured window are removed from the front of the deque.

8.3 Token Bucket

The TokenBucketLimiter stores the current token count and the last refill time. Tokens are refilled according to the elapsed simulated time, up to the bucket capacity. A request is accepted when at least one token is available.

8.4 Leaky Bucket

The LeakyBucketLimiter stores the current bucket level and the last update time. The bucket level decreases according to the elapsed simulated time. A new request is accepted if the bucket has remaining capacity.

8.5 Traffic Generator

TrafficGenerator converts configured rates into simulated timestamps. Normal and overload traffic use a constant rate, while burst and periodic patterns change the rate during the simulation. Random traffic selects varying intervals between configured bounds.

8.6 Benchmarking

BenchmarkEngine resets the limiter, processes every generated request, counts accepted and rejected requests, and measures the Java processing time using System.nanoTime(). The resulting Processing Events/sec value describes simulator processing speed, not network throughput.

8.7 CLI and Validation

The application provides both interactive mode and command-line mode. Input helper methods validate integer values, while command-line parsing checks algorithm names, traffic patterns, and required argument values.

Complexity Summary

Algorithm	Time / Request	Space
Fixed Window	$O(1)$	$O(1)$
Sliding Window	$O(1)$ amortized	$O(\text{limit})$
Token Bucket	$O(1)$	$O(1)$
Leaky Bucket	$O(1)$	$O(1)$

9. Screenshots / Results :

A sudden-burst experiment was used during development to verify that the simulator produces different outcomes instead of accepting every request.

```

=====
                        RATEFORGE
                    Rate Limiting Experimentation
=====

```

1. Run a single algorithm
2. Compare all algorithms
3. Live request simulation
4. Run experiment preset
5. Learn about algorithms
6. Export last results to CSV
7. Exit

Choose an option: 2

```

=====
                        EXPERIMENT DETAILS
=====

```

```

Rate Limit      : 100 requests/sec
Duration        : 10 seconds
Base Traffic    : 25 requests/sec
Peak Traffic    : 400 requests/sec
Traffic Pattern : BURST
Total Requests  : 551
Window Size     : 1000 ms
=====

```

Running experiment...

- > Fixed Window
- > Sliding Window
- > Token Bucket
- > Leaky Bucket

Benchmark Results

```

-----
Algorithm      Accepted   Rejected   Accept %   Reject %   Processing Events/sec
-----
Fixed Window    400         151       72.60      27.40      2065217.39
Sliding Window  325         226       58.98      41.02      1175090.64
Token Bucket    422         129       76.59      23.41      2138975.16
Leaky Bucket    423         128       76.77      23.23      1412820.51
-----

```

Example experiment configuration:

Parameter	Value
Rate Limit	100 requests/sec
Duration	10 seconds
Base Traffic	20 requests/sec
Peak Traffic	500 requests/sec
Traffic Pattern	BURST
Generated Requests	673

Observed benchmark results from the development run:

Algorithm	Accepted	Rejected	Accept %	Reject %
Fixed Window	360	313	53.49	46.51
Sliding Window	280	393	41.60	58.40
Token Bucket	378	295	56.17	43.83
Leaky Bucket	379	294	56.32	43.68

The experiment demonstrates that the algorithms do not necessarily produce the same result when the same burst workload is applied. The difference comes from how each algorithm treats time windows, stored requests, tokens, or bucket occupancy.

The Processing Events/sec value was intentionally described as simulator processing speed. It should not be presented as API or network throughput because this project does not yet perform real network requests.

10. Testing Approach :

Testing for this was focused on functional behaviour and boundary cases of the individual algorithms, traffic generator, CLI, and result generation.

Algorithm Tests

Test	Expected Behaviour
Requests below limit	Requests should be accepted while capacity is available.
Requests above limit	Excess requests should be rejected.
Window transition	Fixed Window should reset when a new window begins.
Sliding expiration	Old timestamps should be removed from the rolling window.
Token refill	Tokens should increase with simulated elapsed time up to capacity.
Empty token bucket	Requests should be rejected until enough tokens are available.
Full leaky bucket	Requests should be rejected while the bucket is at capacity.
Reset	Limiter state should return to its initial state.

Application Tests

- Run the application without arguments and verify the main menu appears.
- Run each menu option and check that invalid values are rejected.
- Run all experiment presets and verify that requests are generated.
- Compare all algorithms using the same request list.
- Run live simulation and verify request decisions are displayed.
- Export results and verify that `rateforge_results.csv` is created.

- Run --help and --demo from the command line.
- Test invalid algorithm names and invalid traffic patterns.

Test Result Summary

Area	Status
Compilation	Passed
Interactive menu	Passed
Traffic generation	Passed
Single algorithm experiments	Passed
Algorithm comparison	Passed
Live simulation	Passed
Experiment presets	Passed
CSV export	Passed
Input validation	Passed

11. Challenges Faced :

Making traffic meaningful: The first version generated requests using simple timestamp increments. This could result in all algorithms accepting 100% of requests. The traffic generator was redesigned around request rates, duration, and traffic patterns.

Making the CLI useful: A benchmark that automatically runs without user input does not show much of the actual application. An interactive menu, presets, custom experiments, and live mode were added.

Comparing algorithms fairly: Each algorithm needs to receive the same workload for a useful comparison. RateForge generates the request stream once and reuses it for the algorithms.

Understanding simulated performance: The original requests/sec value could be confused with real API throughput. It was renamed to Processing Events/sec and explained as simulator performance.

Handling algorithm state: Each algorithm needs different internal state. The common interface provides the same external operations while allowing each implementation to maintain its own data.

Keeping the project manageable: The project was kept as a single Java file so it remains easy to compile and understand. A multi-file structure is planned for future development.

12. Conclusion :

RateForge provides a practical command-line environment for implementing and experimenting with common rate-limiting algorithms. The project moved from a simple predefined benchmark to an interactive simulator where users can configure traffic and observe algorithm behaviour.

The current version demonstrates Java interfaces, classes, collections, enums, exception handling, algorithm analysis, simulation, and basic software engineering. It also provides repeatable experiment scenarios and CSV output for further analysis.

The project is intentionally limited to simulation in this version. Features such as real HTTP requests, per-user limiting, distributed storage, and a web interface can be added later without changing the main educational purpose of the project.
